

# NPL-AV1: A Neural Pruner Ladder for the AV1 Intra-Frame Partition Search

Pablo Rosa, Daniel Palomino, Marcelo Porto, Luciano Agostini

*Video Technology Research Group (ViTech)*

*Federal University of Pelotas*

Pelotas, Brazil

{pablo.rosa, dpalomino, porto, agostini}@inf.ufpel.edu.br

**Abstract**—In the AOMedia Video 1 (AV1) codec, intra-frame coding selects the block partitioning through a recursive rate-distortion search. At its fast speed presets, the reference encoder prunes this search with a convolutional neural network. This paper presents the Neural Pruner Ladder for AV1 (NPL-AV1), a set of shallow neural networks that replace the native network at the same speed preset, keeping its other speed features unchanged. Instead of one network consulted once per block before the search, NPL-AV1 consults one network per node size at three points of the search. A pre-search pruner removes partition types before any rate-distortion evaluation. A post-NONE pruner decides whether the search continues after the unpartitioned node is measured, a decision that no neural model of the native encoder takes in All-Intra coding. An extended pruner decides whether the extended partition types are evaluated. Over the eight UHD 4K sequences of the AOM Common Test Conditions, the post-NONE pruner equals or improves the coding efficiency of the native network at presets 1 and 2, significantly at high quality, with time savings of 30.34% and a Bjontegaard delta bitrate (BD-BR) of 0.41% at preset 1. The pre-search pruner exceeds the time savings of the native network at every preset, with 40.20% to 77.30%, and the extended pruner adds up to 0.87 percentage points in all eight sequences, reaching 50.99% with a BD-BR of 1.07% at preset 2. Ten of the thirteen nondominated operating points belong to NPL-AV1.

**Index Terms**—Video coding, AV1, computational effort reduction, intra-frame block partitioning, neural networks, encoder speed presets.

## I. INTRODUCTION

The number of Internet users watching online video every month keeps growing [1]. The AOMedia Video 1 (AV1) was released in 2018 by the Alliance for Open Media (AOMedia) [2], alongside the High Efficiency Video Coding (HEVC) [3] and the Versatile Video Coding (VVC) [4] of the ITU-T and ISO/IEC international bodies. Each step of its hybrid coding flow introduces new techniques [5], [6]. Its coding efficiency is superior to that of HEVC under certain conditions [7], at the price of encoding times about twice as long [8].

The processing of the partition tree is one of the large computational costs of the AV1 reference encoder [9]. Each superblock is divided by a recursive search that evaluates up to ten partition types per node through rate-distortion optimization (RDO) [10]. The reference software libaom [11] performs this search in full at its highest-quality speed preset, `cpu-used=0`. From `cpu-used=1` on, its All-Intra configuration prunes this search with a convolutional neural network

(CNN), the CNN-based intra partition pruning of libaom [11]. This CNN reads each  $64 \times 64$  block once and, before any RDO, forces or forbids the quad-tree split down to  $8 \times 8$  samples.

Few works act on the computational effort of AV1. None of them acts on the partition network of its reference encoder. The works [12]–[14] reduce the intra prediction mode decision, [15] applies decision trees to inter prediction, and [16], [17] are hardware designs for the intra predictor rather than for encoder decisions. Neural partition prediction was integrated into the VP9 encoder through a hierarchical CNN [18]. For VVC, learning-based partitioning solutions use variance and gradients, random forests and gradient boosting, texture complexity, or the regression of rate-distortion (RD) costs [19]–[24]. These solutions were designed for the partition structures of those codecs and evaluated on their encoders, which prevents a direct and fair comparison.

This paper presents the Neural Pruner Ladder for AV1 (NPL-AV1), a set of shallow multilayer perceptrons (MLPs) that replace the native CNN in the libaom intra-frame partition search. The main strategy of this solution is to consult each pruner where the information it reads already exists. Instead of one CNN that reads each  $64 \times 64$  block once, NPL-AV1 consults one MLP per node size at three points of the node search. A pre-search pruner acts before any RDO, where the native CNN acts. A post-NONE pruner reads the RD cost of the unpartitioned node and decides whether the search continues, a decision that no neural model of the native encoder takes in All-Intra coding. An extended pruner decides whether the extended partition types of Section II are evaluated. Every pruner is measured at a fixed preset with the native CNN disabled, since a preset also changes the transform, intra mode and loop filter searches: only a replacement at the same preset measures the partition pruner alone. Over the eight UHD 4K sequences of the AOM Common Test Conditions (CTC), the post-NONE pruner equals or improves the coding efficiency of the native CNN at presets 1 and 2, significantly at high quality. The pre-search pruner exceeds the time savings of the native CNN at every preset, and the extended pruner adds time savings in all eight sequences, so ten of the thirteen nondominated operating points belong to NPL-AV1, with models trained on a single public dataset.

## II. THE AV1 INTRA-FRAME PARTITION SEARCH

The basic unit of AV1 partitioning is the superblock, of  $128 \times 128$  or  $64 \times 64$  samples, which the partition tree divides recursively down to  $4 \times 4$  samples [5]. Each node may take one of ten partition types [6]. NONE codes the node as a single block, SPLIT divides it into four square sub-blocks, and HORZ and VERT divide it into two rectangular sub-blocks. The four AB types join a rectangular half to two square sub-blocks, and the two 4-way types divide it into four sub-blocks of 4:1 aspect ratio. Together, the AB and 4-way families are the extended partitions.

The libaom encoder evaluates the types of a node in a fixed order, NONE, SPLIT, the rectangular and then the extended types [11], so what a pruner can read depends on where it is consulted. Before NONE, only the samples, the quantization and the decisions of the already coded neighbors exist; after NONE, the rate, the distortion and the RD cost of the unpartitioned node exist as well, and a pruner consulted there can still save the evaluation of every remaining type.

In All-Intra coding, the native neural pruners leave the point after NONE almost free. Before the search, the CNN of Section I forces or forbids the split. After NONE, three neural decisions can end the search of the node or remove candidates from it: a breakout after NONE, an early termination after SPLIT and the pruning of the rectangular types. A frame-type guard disables all three in intra-only frames [11]. Only two networks stay active after NONE, and they prune the AB and 4-way types from the RD costs already measured. Whether the search of a node continues once its unpartitioned cost is known is, therefore, a decision that no neural model of the native encoder takes in All-Intra coding, and the one that the post-NONE pruner of Section III takes.

This work is focused on the intra-frame partition search of AV1 under the All-Intra (AI) configuration. The intra prediction modes, the transform type and the loop filters are decided inside each evaluated partition type, not by the partition search itself, and therefore they are not focused on this work, nor is the inter-frame partition search.

## III. PROPOSED NPL-AV1 SOLUTION

The Neural Pruner Ladder for AV1 (NPL-AV1) inserts MLP pruners at the two points of the node search identified in Section II, as shown in Fig. 1. The main strategy of this solution is to consult each pruner where the information it reads already exists. At each preset, the native CNN is disabled and replaced by the post-NONE pruner or by the pre-search pruner, the latter alone or with the extended pruner, and every other speed feature of the preset is kept as it is.

The pre-search pruner is consulted where the native CNN is and acts on the same structural axis: it may settle the node as NONE and cut the whole subtree below it, commit the node to SPLIT, or remove the rectangular types and, with them, the extended ones. The post-NONE and the extended pruners are consulted after NONE has been evaluated, a point at which the native encoder takes no structural neural decision in All-Intra coding, so NPL-AV1 uses the RD cost that the native

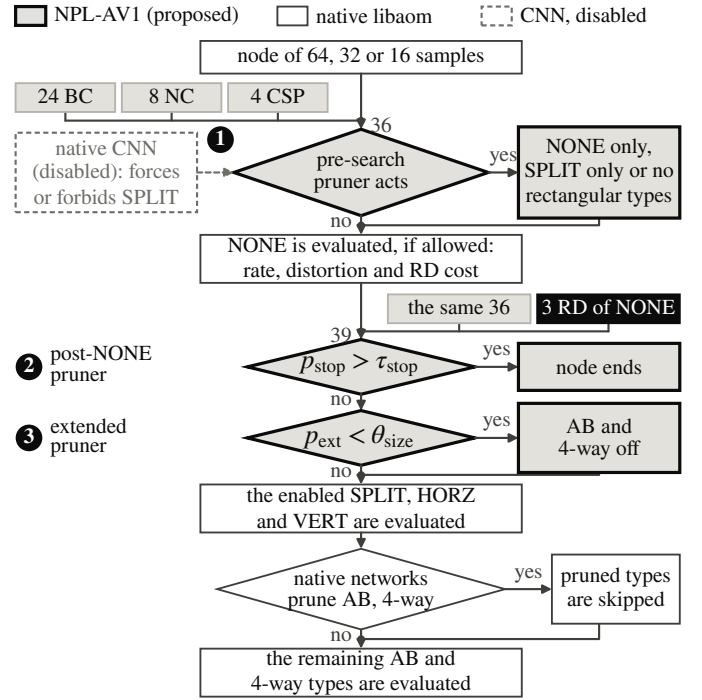


Fig. 1. NPL-AV1 flowchart for one node at a fixed preset. The proposed pruners act at points 1 (pre-search, where the disabled native CNN acted), 2 (post-NONE) and 3 (extended); each operating point enables only some of them, and the native networks of the AB and 4-way types stay active.

CNN never reads. The post-NONE pruner decides whether the search of the node ends there, and the extended pruner whether the AB and 4-way types are worth evaluating. Every pruner only removes candidates, so moving between the operating points of a preset is a matter of choosing which pruner replaces the CNN, not of retraining it.

The rule behind the ladder is that two pruners add their time savings when they remove disjoint sets of candidates. The extended pruner removes only AB and 4-way candidates, and only on the nodes the pre-search pruner did not settle, so its savings are expected to add to those of the pre-search pruner. From preset 3 on, libaom itself prunes the extended partitions from the result of SPLIT, so the gain of the extended pruner is expected at presets 1 and 2, as Section IV confirms.

### A. Features, Dataset and Models

Every pruner reads features whose extraction demands no rate-distortion evaluation of its own. The pre-search pruner reads thirty-six features in three sets. Twenty-four block-content features (BC) are statistics of the samples of the node, of its quadrants and of its contrast with the parent and sibling sub-blocks, together with the quantization and the position of the node. Eight causal-neighbor partition context features (NC) read the partition of the coded blocks above and on the left, and four coding-state and position features (CSP) are added to them. The post-NONE and the extended pruners also read the logarithms of the rate, the distortion and the RD cost of the measured NONE.

The label of every node comes from an extraction with libaom v3.10.0, instrumented to record the decision of the encoder with these features at `cpu-used=0`, since only an exhaustive search gives a faithful label. All sixteen UHD 4K sequences of UVG [25] were encoded over five frames at `cq-level` 20, 32, 43 and 55, with ten sequences kept for training, three for validation and three for testing. The CTC sequences of Section IV do not belong to UVG, which rules out data leakage between training, testing and evaluation.

Every pruner is a set of three MLPs, for nodes of 16, 32 and 64 samples, whose two hidden layers have 64 and 32 ReLU units. The pre-search and the post-NONE pruners give the probabilities of NONE, of SPLIT and of the remaining types, and the extended pruner gives the probability  $p_{\text{ext}}$  that the best type of the node is an extended one.

### B. Decision Rules and Encoder Integration

The pre-search pruner reads its probabilities in a fixed order: the node is settled as NONE, with the whole subtree below it cut, when  $p_{\text{none}} > \tau_{\text{none}}$ ; otherwise it is committed to SPLIT when  $p_{\text{split}} > \tau_{\text{split}}$ ; otherwise its rectangular types are disabled, together with the extended ones, when  $p_{\text{rest}} < \tau_{\text{rest}}$ . The post-NONE pruner ends the search of the node when its probability of NONE,  $p_{\text{stop}}$ , exceeds  $\tau_{\text{stop}} = 0.95$ , and the extended pruner disables the AB and 4-way types when  $p_{\text{ext}} < \theta_{\text{size}}$ , with  $\theta_{\text{size}} = 0.0910$  for nodes of 16 samples, 0.1031 for 32 and 0.0144 for 64.

A higher  $\tau_{\text{none}}$  or  $\tau_{\text{split}}$ , or a lower  $\tau_{\text{rest}}$ , demands more confidence from the model before a candidate is removed, so fewer wrong decisions are taken and more coding efficiency is kept. Loosening the three thresholds moves the pruner toward larger time savings. Two operating points were selected at opposite sides of this trade-off, from the same models and without retraining. Set by  $(\tau_{\text{none}}, \tau_{\text{split}}, \tau_{\text{rest}})$ , the efficiency-first one, (0.95, 0.90, 0.20), is stricter in all three thresholds, and the time-first one, (0.60, 0.85, 0.40), is looser in all three.

All pruners were implemented in C within libaom v3.10.0. With their compile-time guard disabled, the build gives the bitstream of the reference software byte for byte. The native CNN is disabled in the same binary, so at each preset the partition pruner is the only difference from the native configuration. Since every pruner removes candidates and never forces one, NPL-AV1 is fully compliant with the AV1 specification and does not affect the bitstream syntax.

## IV. RESULTS AND COMPARISONS

NPL-AV1 was evaluated in the libaom v3.10.0 reference software under the All-Intra configuration of the AOM CTC [26]. The eight sequences of Class A1 have  $3840 \times 2160$  samples, 10-bit depth and 4:2:0 chroma subsampling [27]. Each sequence was encoded over its first 15 frames at `cq-level` 20, 32, 43 and 55, with the two-tile setting of the CTC for 4K content, against libaom at `cpu-used=0` as anchor. Three deviations from the CTC are declared. The quantization is set by `cq-level`, since this version of libaom has no `--qp` option, and the time is the wall time of each encode. The

TABLE I  
BD-BR AND TS AGAINST LIBAOM AT `CPU-USED=0`: NATIVE CNN AND NPL-AV1 PRUNERS AT PRESETS 1 TO 3

| Partition pruner                    | Preset 1  |        | Preset 2  |        | Preset 3  |        |
|-------------------------------------|-----------|--------|-----------|--------|-----------|--------|
|                                     | BD-BR (%) | TS (%) | BD-BR (%) | TS (%) | BD-BR (%) | TS (%) |
| libaom native CNN                   | 0.449     | 32.59  | 0.536     | 42.72  | 2.722     | 67.94  |
| NPL-AV1:                            |           |        |           |        |           |        |
| Post-NONE                           | 0.414     | 30.34  | 0.516     | 40.73  | 3.384     | 70.25  |
| Pre-search,                         |           |        |           |        |           |        |
| efficiency-first                    | 0.915     | 40.20  | 1.030     | 50.05  | 3.866     | 73.09  |
| time-first                          | 1.685     | 51.82  | 1.805     | 60.97  | 4.347     | 77.30  |
| Extended pruner, separate campaign: |           |        |           |        |           |        |
| efficiency-first                    | 0.915     | 40.28  | 1.030     | 50.16  | 3.866     | 73.10  |
| + extended pruner                   | 0.947     | 41.15  | 1.069     | 50.99  | 3.868     | 72.96  |

configurations under evaluation run at presets 1, 2 and 3, while only the anchor keeps `cpu-used=0`, since the preset is the controlled variable of this work. The Bjontegaard delta bitrate (BD-BR) [28] uses the luma peak signal-to-noise ratio (PSNR), and a positive value is a coding efficiency loss. With  $t(s, q)$  and  $t_{\text{anc}}(s, q)$  the encoding times of a configuration and of the anchor for sequence  $s$  at `cq-level`  $q$ , and  $S$  and  $Q$  the sets of sequences and of `cq-level` values, the time savings (TS) are

$$\text{TS} = \frac{100}{|S|} \sum_{s \in S} \frac{1}{|Q|} \sum_{q \in Q} \left( 1 - \frac{t(s, q)}{t_{\text{anc}}(s, q)} \right) \%. \quad (1)$$

Each configuration was encoded once. Five repeated encodes on one sequence give 0.23 percentage points of standard deviation for TS, so TS differences under 0.46 points are not resolved; the BD-BR comes from the bitstream size and the PSNR, which are exact.

Table I gives both metrics for the native CNN and for the NPL-AV1 pruners that replace it at each preset. At presets 1 and 2, the post-NONE pruner matches the coding efficiency of the native CNN, with 0.414% against 0.449% of BD-BR at preset 1 and 0.516% against 0.536% at preset 2, for 30.34% and 40.73% of TS. The paired t-tests of Table II, over the eight sequences, show no significant BD-BR difference at either preset. Table III shows the comparison sequence by sequence: the post-NONE pruner has the lower BD-BR in nine of the sixteen cases and in both averages. Preset 3 is the only one at which the native CNN keeps the lower BD-BR.

The average over the whole grid is the mean of two regimes, and in both the post-NONE pruner equals or improves the coding efficiency of the native CNN. At `cq-level` 20 and 32, it lowers the BD-BR of the native CNN by 0.088 points at preset 1 and by 0.086 at preset 2, both significant in Table II ( $p = 0.043$  and  $p = 0.015$ ), in six and seven of the eight sequences. At `cq-level` 43 and 55, the two pruners do not differ in BD-BR or in TS. The split by quantization regime was defined after the whole-grid result, and it is replicated with the same sign and magnitude at two independent presets. This fact partly explains why a shallow model reaches the coding efficiency of the native network: it reads the measured RD

TABLE II  
POST-NONE PRUNER COMPARED WITH THE NATIVE CNN

| cq-level                                             | BD-BR         |              | Seq. | TS            |              |
|------------------------------------------------------|---------------|--------------|------|---------------|--------------|
|                                                      | $\Delta$ (pp) | $p$          |      | $\Delta$ (pp) | $p$          |
| <i>Preset 1</i>                                      |               |              |      |               |              |
| 20 to 55                                             | -0.034        | 0.278        | 4    | -2.25         | <b>0.046</b> |
| 20, 32                                               | -0.088        | <b>0.043</b> | 6    | -4.29         | <b>0.036</b> |
| 43, 55                                               | +0.009        | 0.826        | 3    | -0.22         | 0.678        |
| <i>Preset 2</i>                                      |               |              |      |               |              |
| 20 to 55                                             | -0.020        | 0.291        | 5    | -1.99         | 0.055        |
| 20, 32                                               | -0.086        | <b>0.015</b> | 7    | -3.88         | <b>0.041</b> |
| 43, 55                                               | +0.027        | 0.467        | 3    | -0.10         | 0.859        |
| <i>Preset 3</i>                                      |               |              |      |               |              |
| 20 to 55                                             | +0.662        | <b>0.004</b> | 0    | +2.30         | <b>0.002</b> |
| Seq.: sequences with lower BD-BR. Bold: $p < 0.05$ . |               |              |      |               |              |

TABLE III  
PER-SEQUENCE BD-BR AND TS AT PRESETS 1 AND 2

| Sequence       | Preset 1            |                     | Preset 2            |                     |
|----------------|---------------------|---------------------|---------------------|---------------------|
|                | Native              | Post-NONE           | Native              | Post-NONE           |
| BoxingPractice | 0.587/33.47         | <b>0.554</b> /33.00 | <b>0.600</b> /42.34 | 0.637/41.80         |
| Crosswalk      | 0.093/32.78         | <b>0.061</b> /33.02 | 0.530/46.30         | <b>0.498</b> /46.63 |
| FoodMarket2    | <b>0.170</b> /29.18 | 0.171/26.58         | <b>0.184</b> /38.79 | 0.203/36.48         |
| Neon1224       | 0.725/35.60         | <b>0.520</b> /31.42 | 0.773/43.90         | <b>0.649</b> /40.25 |
| NocturneDance  | <b>0.817</b> /28.66 | 0.864/26.92         | 0.888/39.36         | <b>0.854</b> /37.45 |
| PierSeaSide    | 0.481/35.88         | <b>0.382</b> /31.60 | 0.436/46.91         | <b>0.423</b> /43.14 |
| Tango          | <b>0.348</b> /32.46 | 0.383/33.88         | <b>0.437</b> /42.47 | 0.454/44.21         |
| TimeLapse      | <b>0.369</b> /32.72 | 0.378/26.31         | 0.440/41.71         | <b>0.409</b> /35.89 |
| Average        | 0.449/32.59         | <b>0.414</b> /30.34 | 0.536/42.72         | <b>0.516</b> /40.73 |

Each cell: BD-BR (%) / TS (%). Bold: the lower BD-BR of the preset.

cost of NONE, which no native network reads before deciding whether the search of the node continues.

The pre-search pruner exceeds the time savings of the native CNN at every preset. At the efficiency-first point it lifts the TS of the native CNN by 5.15 to 7.61 percentage points, at a BD-BR 1.4 to 2.0 times higher, and the time-first point reaches 77.30% of TS at preset 3, a speedup of 4.46 times, at 4.347% of BD-BR.

The extended pruner confirms the rule of Section III. Against the efficiency-first point re-encoded in its own campaign, it lifts the TS by 0.87 percentage points at preset 1 and by 0.83 at preset 2, in all eight sequences ( $p = 0.001$ ), at 0.032 and 0.039 points of BD-BR. At preset 2, it lifts the TS of that re-encoded point from 50.16% to 50.99%, with 1.069% of BD-BR. This is 27 and 21 points of TS per point of BD-BR, 1.8 and 1.5 times the 15.1 and 14.1 obtained when the thresholds of the pre-search pruner are loosened from the efficiency-first to the time-first point. As Section III anticipates, the increment falls below the resolution of 0.46 points at preset 3, where libaom already prunes the extended partitions.

One more NPL-AV1 point was measured at `cpu-used=0`, with the pre-search pruner at (0.90, 0.90),  $\tau_{\text{rest}}$  disabled, and the post-NONE pruner: 0.160% of BD-BR for 12.61% of TS, the lowest BD-BR of all configurations. Among the sixteen configurations measured, the twelve of Table I, the three with the extended pruner and the one at `cpu-used=0`, thirteen are nondominated in BD-BR and TS, and ten of them belong

to NPL-AV1. At every preset, NPL-AV1 adds points on both sides of the native CNN: the post-NONE pruner at its coding efficiency, at presets 1 and 2, and the pre-search pruner beyond its time savings.

The cost of the pruners themselves was measured in the encoder over three frames of two sequences at preset 1, and every NPL-AV1 pruner takes less than 0.5% of the encoding time: 0.26% to 0.36% for the post-NONE pruner, 0.41% to 0.48% for the efficiency-first pre-search pruner and 0.37% to 0.42% for the extended pruner. The time savings come, therefore, from the removed candidates and not from the inference. Since no work replaces the partition network of the AV1 reference encoder, the native CNN is the reference of comparison of this work.

## V. CONCLUSIONS

This paper presented NPL-AV1, a set of shallow neural pruners that replace the native CNN of the AV1 intra-frame partition search at the same speed preset, so that each preset of libaom gains operating points on both sides of the native CNN. Instead of one CNN that reads each  $64 \times 64$  block once, NPL-AV1 consults one MLP per node size at three points of the node search: before it, after NONE, where no neural model of the native encoder decides in All-Intra coding whether the search ends, and before the AB and 4-way types. The pruners were implemented in C within libaom v3.10.0 and evaluated over the eight CTC Class A1 sequences at presets 1, 2 and 3. At presets 1 and 2, the post-NONE pruner equals or improves the coding efficiency of the native CNN in all six comparisons of Table II, significantly at high quality, with 0.41% of BD-BR for 30.34% of time savings at preset 1. The pre-search pruner exceeds the time savings of the native CNN at every preset, up to 77.30%. The extended pruner adds 0.87 and 0.83 percentage points in all eight sequences and, with the pre-search pruner, reaches 50.99% of time savings with 1.07% of BD-BR at preset 2. Ten of the thirteen nondominated operating points belong to NPL-AV1. These results come from MLPs with two hidden layers trained on a single public dataset, five frames of each of the sixteen UVG sequences, which indicates that the architecture of NPL-AV1 already equals or improves the native CNN with a small training budget. Larger and more diverse training sets and deeper models at the same three points of the search are, therefore, a direction for future work. Besides, to the best of the authors' knowledge, this is the first work that measures a replacement of the native partition network of AV1 at the same preset.

## ACKNOWLEDGMENT

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001, and also by the Brazilian research support agencies CNPq and FAPERGS. The authors used Claude (Anthropic) for English language proofreading and grammar checking in the paper. All content was reviewed and edited by the authors, who take full responsibility for the final work.

## REFERENCES

- [1] Statista. (2025). *Share of Internet Users Watching Online Videos Every Month From 1st Quarter 2022 to 3rd Quarter 2024*. [Online]. Available: <https://www.statista.com/statistics/1489445/online-video-viewers-worldwide-quarterly/>
- [2] Alliance for Open Media. (2019). *AV1 Video Codec*. [Online]. Available: <https://aomedia.org/av1>
- [3] ITU-T. (2013). *H.265: High Efficiency Video Coding*. [Online]. Available: <https://www.itu.int/rec/t-rec-h.265>
- [4] ITU-T. (2020). *H.266: Versatile Video Coding*. [Online]. Available: <https://www.itu.int/rec/t-rec-h.266>
- [5] J. Han *et al.*, “A technical overview of AV1,” *Proc. IEEE*, vol. 109, no. 9, pp. 1435–1462, Sep. 2021.
- [6] Y. Chen *et al.*, “An overview of core coding tools in the AV1 video codec,” in *Proc. Picture Coding Symp. (PCS)*, Jun. 2018, pp. 41–45.
- [7] D. Grois, T. Nguyen, and D. Marpe, “Coding efficiency comparison of AV1/VP9, H.265/MPEG-HEVC, and H.264/MPEG-AVC encoders,” in *Proc. Picture Coding Symp. (PCS)*, Dec. 2016, pp. 1–5.
- [8] M. A. Layek *et al.*, “Performance analysis of H.264, H.265, VP9 and AV1 video encoders,” in *Proc. 19th Asia-Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sep. 2017, pp. 322–325.
- [9] I. Bender *et al.*, “Complexity and compression efficiency analysis of libaom AV1 video codec,” *J. Real-Time Image Process.*, vol. 20, no. 3, art. no. 50, May 2023.
- [10] G. J. Sullivan and T. Wiegand, “Rate-distortion optimization for video compression,” *IEEE Signal Process. Mag.*, vol. 15, no. 6, pp. 74–90, Nov. 1998.
- [11] Alliance for Open Media. *AV1 Codec Library*, v3.10.0. [Online]. Available: <https://aomedia.googlesource.com/aom>
- [12] M. Xu and B. Jeon, “Selection of intra prediction tools for fast AV1 encoding,” in *Proc. IEEE Int. Symp. Broadband Multimedia Syst. Broadcast. (BMSB)*, Oct. 2020, pp. 1–4.
- [13] M. Xu and B. Jeon, “User-priority based AV1 coding tool selection,” *IEEE Trans. Broadcast.*, vol. 67, no. 3, pp. 736–745, Sep. 2021.
- [14] J. Jeong, G. Gankhuyag, and Y.-H. Kim, “A fast intra mode decision based on accuracy of rate distortion model for AV1 intra encoding,” in *Proc. 34th Int. Tech. Conf. Circuits/Syst., Comput. Commun. (ITC-CSCC)*, Jun. 2019, pp. 1–3.
- [15] J. Kim, S. Blasi, A. S. Dias, M. Mrak, and E. Izquierdo, “Fast inter-prediction based on decision trees for AV1 encoding,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 1627–1631.
- [16] M. M. Corrêa *et al.*, “A high-throughput hardware architecture for AV1 non-directional intra modes,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 5, pp. 1481–1494, May 2020.
- [17] L. Neto, M. Corrêa, D. Palomino, L. Agostini, and G. Corrêa, “Power-quality configurable hardware design for AV1 directional intraframe prediction,” *IEEE Des. Test*, vol. 39, no. 2, pp. 38–45, Apr. 2022.
- [18] S. Paul, A. Norkin, and A. C. Bovik, “Speeding up VP9 intra encoder with hierarchical deep learning-based partition prediction,” *IEEE Trans. Image Process.*, vol. 29, pp. 8134–8148, 2020.
- [19] J. Chen, H. Sun, J. Katto, X. Zeng, and Y. Fan, “Fast QTMT partition decision algorithm in VVC intra coding based on variance and gradient,” in *Proc. IEEE Vis. Commun. Image Process. (VCIP)*, Dec. 2019, pp. 1–4.
- [20] Q. He, W. Wu, L. Luo, C. Zhu, and H. Guo, “Random forest based fast CU partition for VVC intra coding,” in *Proc. IEEE Int. Symp. Broadband Multimedia Syst. Broadcast. (BMSB)*, Aug. 2021, pp. 1–4.
- [21] M. Saldanha, G. Sanchez, C. Marcon, and L. Agostini, “Configurable fast block partitioning for VVC intra coding using light gradient boosting machine,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 6, pp. 3947–3960, Jun. 2022.
- [22] Y. Song, S. Cheng, M. Wang, and X. Peng, “Fast CU partition for VVC intra-frame coding via texture complexity,” *IEEE Signal Process. Lett.*, vol. 31, pp. 959–963, 2024.
- [23] M. E. A. Kherchouche, F. Galpin, T. Dumas, D. Menard, and L. Zhang, “RD-cost regression speed up technique for VVC intra block partitioning,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2024, pp. 3530–3534.
- [24] M. E. A. Kherchouche *et al.*, “Complexity reduction study based on RD costs approximation for VVC intra partitioning,” in *Proc. Data Compression Conf. (DCC)*, Mar. 2025, p. 377.
- [25] A. Mercat, M. Viitanen, and J. Vanne, “UVG dataset: 50/120fps 4K sequences for video codec analysis and development,” in *Proc. 11th ACM Multimedia Syst. Conf. (MMSys)*, Jun. 2020, pp. 297–302.
- [26] X. Zhao *et al.*, “AOM common test conditions v9.0,” Alliance for Open Media, Codec Working Group, doc. CWG-G082, Mar. 2026.
- [27] Xiph.Org Foundation. *AOM CTC Test Set, Class A1 (4K)*. [Online]. Available: [https://media.xiph.org/video/aomctc/test\\_set/a1\\_4k/](https://media.xiph.org/video/aomctc/test_set/a1_4k/)
- [28] G. Bjøntegaard, “Calculation of average PSNR differences between RD-curves,” Video Coding Experts Group (VCEG), doc. VCEG-M33, Mar. 2001.